

Acceleration of full-batch algorithms for finite sum problems through parallel computation

Marco Trambusti
Matricola - 7135350
marco.trambusti1@edu.unifi.it

Abstract

The purpose of the paper is to analyze full-batch algorithms for finite sum problems, implementing in Python the **logistic regression** with L2 regularization, with methods for loading datasets and for executing gradient descent with Armijo line search and the conjugate gradient method with Wolfe line search; monitoring execution times and the number of iterations. An initial sequential version has been parallelized, analyzing the acceleration as a function of the number of threads used and the size of the dataset. The parallelization focuses on calculating the loss function and its gradient.

1. Introduction

The **finite sum optimization problems** are common in machine learning, as many loss functions are expressed as a sum of independent terms:

$$\min_{x \in \mathbb{R}^n} f(x) = \frac{1}{N} \sum_{i=1}^N f_i(x). \quad (1)$$

A typical example is **logistic regression**, formulated as:

$$\min_{w \in \mathbb{R}^p} \mathcal{L}(w) + \lambda \Omega(w), \quad \mathcal{L}(w) = \sum_{i=1}^n \log(1 + \exp(-y^{(i)} w^T x^{(i)})). \quad (2)$$

This is an **unconstrained nonlinear problem**, tackled with two methods: **Gradient Descent** (GD) with Armijo line search and **Conjugate Gradient** (CG) with Wolfe conditions. Both ensure convergence, but with different characteristics.

Since each term of the function is independent, the calculation of the objective function and gradient is naturally parallelizable, which is crucial for large datasets.

1.1. Logistic Regression with L2 Regularization

The model with L2 regularization is:

$$\mathcal{L}_{\log}(w) + \lambda \Omega(w) = \sum_{i=1}^n \log(1 + \exp(-y^{(i)} w^T x^{(i)})) + \frac{\lambda}{2} \sum_{j=1}^d w_j^2, \quad (3)$$

with gradient:

$$\nabla_w \mathcal{L}(w) = X^T r + \lambda w, \quad r_i = -y^{(i)} \sigma(-y^{(i)} w^T x^{(i)}). \quad (4)$$

1.2. Gradient Descent (Armijo)

GD updates the weights as:

$$w_{k+1} = w_k - \alpha_k \nabla f(w_k), \quad (5)$$

where α_k is chosen using Armijo backtracking:

$$f(w_k - \alpha \nabla f(w_k)) \leq f(w_k) - \gamma \alpha \|\nabla f(w_k)\|^2. \quad (6)$$

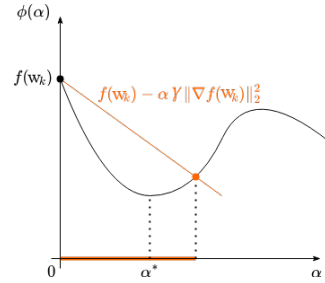


Figure 1. Armijo line search

Algorithm 1 Gradient Descent with Armijo Line Search

- 1: $x \leftarrow x_0 \in \mathbb{R}^n; k \leftarrow 0$
 - 2: **while** $\|\nabla f(x^k)\| > \epsilon$ **do**
 - 3: $d_k \leftarrow -\nabla f(x^k)$
 - 4: Compute α_k using Armijo
 - 5: $x^{k+1} \leftarrow x^k - \alpha_k \nabla f(x^k)$
 - 6: $k \leftarrow k + 1$
 - 7: **end while**
-

1.3. Conjugate Gradient (Wolfe)

CG combines the gradient with previous directions:

$$d_{k+1} = -\nabla f(w_{k+1}) + \beta_k d_k, \quad \beta_k = \frac{\|\nabla f(w_{k+1})\|^2}{\|\nabla f(w_k)\|^2}, \quad (7)$$

with step size α_k satisfying Wolfe conditions:

$$f(w_k + \alpha_k d_k) \leq f(w_k) + \gamma \alpha_k \nabla f(w_k)^T d_k, \quad (8)$$

$$|\nabla f(w_k + \alpha_k d_k)^T d_k| \leq \sigma |\nabla f(w_k)^T d_k|. \quad (9)$$

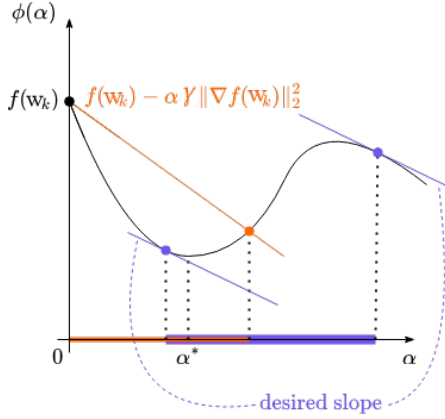


Figure 2. Wolfe conditions

Algorithm 2 Conjugate Gradient with Wolfe Line Search

- 1: $x^0 \in \mathbb{R}^n, d_0 \leftarrow -g_0$
 - 2: **while** $\|g_k\| > \varepsilon$ **do**
 - 3: Compute α_k using Wolfe
 - 4: $x^{k+1} \leftarrow x^k + \alpha_k d_k$
 - 5: $g_{k+1} \leftarrow \nabla f(x^{k+1})$
 - 6: $\beta_{k+1} \leftarrow \frac{\|g_{k+1}\|^2}{\|g_k\|^2}$
 - 7: $d_{k+1} \leftarrow -g_{k+1} + \beta_{k+1} d_k$
 - 8: **end while**
-

1.4. Remarks

GD is simple but slow on ill-conditioned problems; CG converges faster by exploiting conjugate directions, at the cost of a more complex line search. In both cases, parallelizing the computation of loss and gradient is essential for efficiency on large datasets.

2. Implementation

The implementation was developed in Python, using the NumPy library for efficient linear algebra operations. However, summations were computed manually instead of using `numpy.sum` to avoid automatic parallelization

introduced by underlying libraries such as OpenBLAS, which could interfere with control over the computational flow.

For optimizing the regularization parameter λ , cross-validation was employed using the `LogisticRegression` class from `scikit-learn`, configured with L2 regularization. The optimal λ values obtained were stored in the file `cv_results.csv`, making them easily reusable for subsequent experimental sessions or model comparisons.

2.1. Data Representation

Data is loaded from files in the `Datasets` folder using the `load_data` method, which relies on the `load_svmlight_file` function. This function imports the feature matrix X in dense format, where each row represents a sample and each column a descriptive variable. Labels y are loaded into a vector, encoded as binary values in the set $\{-1, 1\}$.

If necessary, the matrix X is normalized to ensure consistent scaling across features. Finally, both X and y are converted to `float32` to optimize computational efficiency for subsequent processing and GPU parallelization.

2.2. Parameters

In `main.py`, key parameters are initialized to ensure proper functioning of line-search strategies according to Armijo and Wolfe criteria. The main ones are:

- **Regularization parameter** λ : determined via cross-validation.
- **Tolerance**: set to 10^{-4} , used as the stopping criterion for convergence.
- **Armijo parameters**: $\gamma = 10^{-4}$, $\delta = 0.5$, where γ controls sufficient decrease and δ adjusts step reduction.
- **Wolfe parameters**: $\gamma = 10^{-4}$, $\delta = 0.8$, $\sigma = 0.2$, with σ imposing a curvature condition to ensure descent direction quality.

2.3. Sequential LogRegressionL2

The `LogisticRegressionL2` class in `log_regression_l2.py` implements logistic regression with L2 regularization. At each iteration, both the objective function (loss) and its gradient are computed using explicit NumPy array operations to avoid automatic parallelization by libraries like OpenBLAS.

Loss computation is handled by the `compute_loss` function, which includes the regularization term:

```

1 def compute_loss(self, X, y, w):
2     ...
3     logits = y * np.dot(X, w)
4     loss_terms = np.logaddexp(0, -logits)
5     total_loss = 0.0
6     for i in range(len(loss_terms)):
7         total_loss += loss_terms[i]
8
9     reg = 0.0
10    for wi in w:
11        reg += wi * wi
12    regularization = (self.lambda_reg / 2) * reg
13    loss = total_loss + regularization
14    ...

```

The gradient is computed in `compute_loss_gradient`, implementing the derivative of the regularized logistic loss:

```

1 def compute_loss_gradient(self, X, y, w):
2     ...
3     logits = X.dot(w)
4     z = -y * logits
5     sig = self.sigmoid(z)
6     r = -y * sig
7     grad = X.T.dot(r) + self.lambda_reg * w
8     ...

```

Line searches based on Armijo and Wolfe criteria are implemented as dedicated methods (`armijo_line_search` and `wolfe_line_search`), each with configurable parameters. Both methods update weights w only if sufficient decrease (Armijo) or curvature (Wolfe) conditions are satisfied.

Execution time for loss and gradient computation is monitored separately and accumulated in `self.total_time`, useful for performance analysis.

2.4. Conjugate Gradient with Wolfe

The conjugate gradient method implemented in the `LogisticRegressionL2` class achieves faster convergence than standard gradient descent by leveraging previous directions to build more effective descent paths. At each iteration, the direction is updated using the Fletcher-Reeves formula.

Line search is performed using Wolfe conditions, ensuring both sufficient decrease of the objective function and adequate curvature along the descent direction. The step size α_k is accepted only if both conditions are met.

```

1 if (loss_new
2     <= current_loss+gamma*alpha*np.dot(grad, direction)
3     and np.abs(np.dot(grad_new, direction))
4     <= sigma*np.abs(np.dot(grad, direction))):
5     break
6 break

```

The core loop of the conjugate gradient method is implemented as follows:

```

1 lr = self.wolfe_line_search(grad, X, y, direction)
2 self.w = np.add(self.w, lr * direction)
3 grad_new = self.compute_loss_gradient(X, y, self.w)
4 beta = np.dot(grad_new, grad_new) / np.dot(grad, grad)
5 direction = -grad_new + beta * direction

```

The `conjugate_gradient_wolfe` method terminates when the gradient norm falls below the predefined tolerance threshold.

3. Parallelization

3.1. Parallelization Points

To accelerate the computation of L2 logistic regression, a parallel version of the code was developed using Numba CUDA, a library that allows writing GPU kernels directly in Python. Numba compiles Python functions into machine code executable on the GPU, enabling fine-grained parallelization without resorting to languages like CUDA C++.

In the context of logistic regression, the most expensive operations—such as loss and gradient computation—were split into CUDA kernels. Each kernel is designed to exploit the independent structure of the data: each thread processes a sample or a feature, and partial results are aggregated using reduction techniques.

For example, the `compute_loss_cuda_kernel_global` kernel computes the logistic loss for each sample:

```

1 @cuda.jit(fastmath=True)
2 def compute_loss_cuda_kernel_global(X_flat, y, w, loss_terms, n_samples,
3     tx = cuda.threadIdx.x
4     bx = cuda.blockIdx.x
5     bw = cuda.blockDim.x
6     grid_size = cuda.gridDim.x
7
8     for i in range(bx, n_samples, grid_size):
9         partial_dot = 0.0
10        for j in range(tx, n_features, bw):
11            index = i * n_features + j
12            partial_dot += X_flat[index] * w[j]
13
14        dot = block_reduce_sum(partial_dot)
15
16        if tx == 0:
17            y_i = y[i]
18            z = -y_i * dot
19            loss = z + math.log1p(math.exp(-z)) if z > 0 \
20                else math.log1p(math.exp(z))
21            loss_terms[i] = loss

```

The `@cuda.jit(fastmath=True)` decorator enables aggressive numerical optimizations, such as faster versions of mathematical functions (e.g., `exp`, `log1p`), which can improve performance at the cost of minimal precision loss. This is particularly useful in machine learning contexts where absolute precision is not always critical.

To aggregate partial results computed by each thread, two auxiliary kernels are used: `warp_reduce_sum` and `block_reduce_sum`. These kernels implement hierarchical reduction within each CUDA block, leveraging shared memory and synchronization primitives.

```

1 @cuda.jit(device=True)
2 def warp_reduce_sum(val):
3     mask = cuda.warpsize // 2
4     while mask > 0:
5         val += cuda.shfl_xor_sync(0xFFFFFFFF, val, mask)
6         mask //= 2
7     return val

```

```

1 @cuda.jit(device=True)
2 def block_reduce_sum(val):
3     warp_size = cuda.warp_size
4     num_warps = cuda.blockDim.x // warp_size
5     shared = cuda.shared.array(shape=32, dtype=float64)
6
7     lane = cuda.threadIdx.x % warp_size
8     wid = cuda.threadIdx.x // warp_size
9
10    val = warp_reduce_sum(val)
11
12    if lane == 0 and wid < num_warps:
13        shared[wid] = val
14    cuda.syncthreads()
15
16    val = 0.0
17    if wid == 0 and lane < num_warps:
18        val = shared[lane]
19    val = warp_reduce_sum(val)
20    return val

```

These kernels are used in multiple parts of the code, for example to sum contributions from dot products or compute the L2 norm of weights. Reduction is essential to obtain a single aggregated value from a set of threads operating in parallel.

The gradient is computed in two main steps:

1. The `compute_r_kernel` computes the vector $r_i = -y_i \cdot \sigma(-y_i \cdot x_i^T w)$ for each sample.
2. The `compute_XTr_kernel` performs the multiplication $X^T r$ in parallel, assigning each thread to a feature.

```

1 @cuda.jit
2 def compute_r_kernel(X_flat, y, w, r, n_samples, n_features):
3     i = cuda.grid(1)
4     if i < n_samples:
5         dot = 0.0
6         base = i * n_features
7         for j in range(n_features):
8             dot += X_flat[base + j] * w[j]
9         z = -y[i] * dot
10        r[i] = -y[i] * sigmoid(z)

```

```

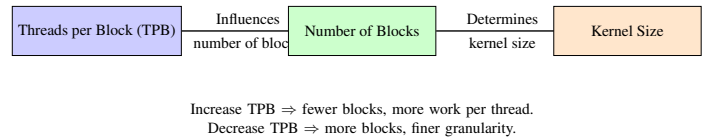
1 @cuda.jit
2 def compute_XTr_kernel(X_flat, r, grad, n_samples, n_features):
3     j = cuda.grid(1)
4     if j < n_features:
5         acc = 0.0
6         for i in range(n_samples):
7             acc += X_flat[i * n_features + j] * r[i]
8         grad[j] = acc

```

The L2 regularization term λw is not implemented as a GPU kernel. It is more efficient to perform this operation directly on the CPU, leveraging its simplicity and avoiding additional data transfers between host and device.

The effective kernel size varies based on the chosen `threadsperblock` (TPB) parameter: increasing TPB reduces the number of blocks required, while lower values increase parallelism granularity. This balance is crucial to fully utilize GPU resources and minimize idle time.

3.2. Kernel Size Diagram



3.3. Warmup: Kernel Pre-Initialization

A key aspect of the parallel implementation is the `warmup_all` method, designed to perform a "pre-activation" of CUDA kernels. This approach reduces the initial latency caused by Numba's Just-In-Time (JIT) compilation.

Numba CUDA compiles kernels at their first execution, introducing significant overhead during the initial call. To prevent this from negatively impacting timing measurements or perceived performance during optimization, the `warmup_all` method executes a series of artificial calls on dummy data:

```

1 X_dummy = np.random.randn(N, D).astype(np.float32)
2 y_dummy = np.random.choice([-1, 1], size=N).astype(np.float32)
3 w_dummy = np.random.randn(D).astype(np.float64)
4
5 compute_loss_cuda_kernel_global...
6 compute_r_kernel...
7 compute_XTr_kernel...

```

These calls force compilation and loading of kernels onto the GPU, ensuring that subsequent executions on real data are immediate and free from initialization overhead.

Additionally, the warmup step allows for early verification of the CUDA pipeline (memory allocation, block sizing, type compatibility), reducing the risk of runtime errors during actual optimization.

4. Results and Performance Evaluation

4.1. Setup

Tests were conducted on a machine equipped with an Intel Core i7-6700K processor (4 cores, 8 threads) and an NVIDIA GeForce GTX 970 GPU (1664 CUDA cores, 4GB memory). Datasets from this site of increasing size were used to assess scalability. The sequential version was compared with parallel versions using 32, 64, 128, 256, 512, and 1024 threads. To validate the optimization methods, results were compared with `scikit-learn`'s Logistic Regression using L2 regularization. Both Gradient Descent and Conjugate Gradient produced consistent final loss values and gradient norms, confirming correctness.

4.2. Results and Evaluation

Dataset	GD Time(s)	GD It.	CG Time(s)	CG It.
768×8	4.91	766	2.19	41
1000×24	2.88	391	6.20	67
1000×60	0.27	30	0.98	11
1605×119	87.19	3949	51.01	204
3185×122	477.03	7730	336.25	336
11055×68	670.30	4298	380.74	287

Table 1. Execution time and iterations for GD and CG (sequential)

Execution time analysis showed significant improvements in the parallel version, with differences depending on dataset size and dimensionality.

Method	Dataset	TPB	Sp. Total	Time (s)
GD	768×8	256	0.83×	5.88
CG	768×8	256	0.78×	2.81
GD	1000×24	256	0.96×	3.00
CG	1000×24	256	0.98×	6.33
GD	1000×60	256	0.93×	0.29
CG	1000×60	256	1.05×	0.94
GD	1605×119	512	1.47×	59.30
CG	1605×119	512	1.86×	27.49
GD	3185×122	512	3.72×	128.12
CG	3185×122	512	5.05×	66.56
GD	11055×68	32	13.76×	48.72
CG	11055×68	32	9.07×	41.98

Table 2. Configurations with best total speedup

Small datasets (768 samples, 8 features). For small-scale problems, parallelization overhead is evident. Gradient Descent (GD) with Armijo line search required 766 iterations and about 4.91 seconds, while Conjugate Gradient (CG) with Wolfe line search converged in only 41 iterations and about 2.19 seconds. CUDA parallelization did not provide substantial benefits: best-case speedups were marginal ($S_p \approx 0.83$ for GD, 0.78 for CG), and efficiency dropped sharply due to synchronization costs. Confirming that GPU acceleration is not advantageous for small-scale problems.

Medium datasets (1000 samples, 24–60 features). As dimensionality increases, parallelization benefits become more evident. For 24 features, sequential GD took 2.88 s, while CUDA with 256 threads reduced time to 3.00 s ($S_p = 0.96$). Sequential CG took 6.20 s, while CUDA with 256 threads achieved 6.33 s ($S_p = 0.98$). For 60 features, sequential GD was very fast (0.27 s), and CUDA versions achieved similar times, with speedups close to 1. Sequential CG required 0.98 s, while CUDA reduced it to about 0.94 s with 256 threads. In this regime, GPU advantage is modest, but CG converges in fewer iterations than GD.

Large datasets (1605–3185 samples, 119–122 features).

Here, parallelization shows clear benefits. For 1605 samples and 119 features, sequential GD took 87.19 s, while CUDA with 256 threads reduced it to 67.47 s ($S_p = 1.29$). Sequential CG took 51.01 s, while CUDA with 256 threads reduced it to 28.94 s ($S_p = 1.76$). Similarly, for 3185 samples and 122 features, sequential GD took 477.03 s, while CUDA with 256 threads reduced it to 139.11 s ($S_p = 3.43$). Sequential CG took 336.25 s, while CUDA with 256 threads achieved 75.35 s ($S_p = 4.46$). These results confirm the effectiveness of GPU acceleration for large-scale problems.

Very large dataset (11,055 samples, 68 features).

In this case, GPU advantage is remarkable. Sequential GD took 670.30 s, while CUDA with 32 threads reduced it to 48.72 s ($S_p = 13.76$). Sequential CG took 380.74 s, while CUDA with 32 threads reduced it to 41.98 s ($S_p = 9.07$). Both methods achieved excellent convergence, with final loss values consistent across implementations. However, efficiency decreased with very high thread counts (512–1024) due to memory bandwidth saturation and synchronization costs.

4.3. Result Analysis

4.3.1 Execution Time

Total execution time grows linearly in sequential methods, while parallel versions exhibit more favorable scalability. For small datasets, initialization and synchronization overhead can outweigh benefits. However, for large datasets, GPU acceleration is significant.

4.3.2 Speedup

Speedup is defined as:

$$S = \frac{T_{\text{seq}}}{T_{\text{par}}}$$

Method	Dataset	TPB	Sp. Tot	Iterations
GD CUDA	768×8	256	0.83×	766
CG CUDA	768×8	256	0.78×	44
GD CUDA	1000×24	256	0.96×	391
CG CUDA	1000×24	256	0.98×	67
GD CUDA	1000×60	256	0.93×	30
CG CUDA	1000×60	256	1.05×	11
GD CUDA	1605×119	512	1.47×	3949
CG CUDA	1605×119	512	1.86×	149
GD CUDA	3185×122	64	5.40×	7730
CG CUDA	3185×122	512	5.05×	289
GD CUDA	11055×68	32	13.76×	4298
CG CUDA	11055×68	32	9.07×	336

Table 3. Configurations with best total speedup

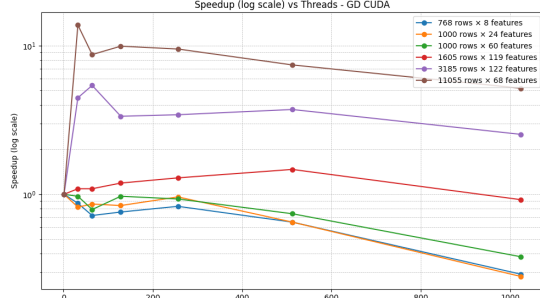


Figure 3. Gradient Descent Total speedup vs TPB

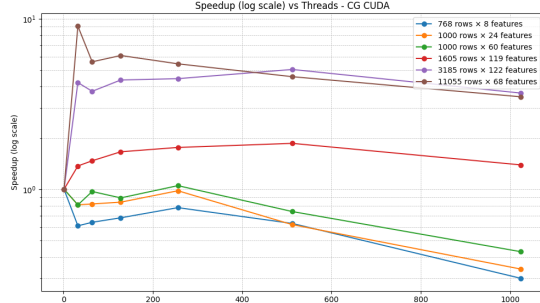


Figure 4. Conjugate Gradient Total speedup vs TPB

Gradient Speedup Analysis Gradient computation shows increasing speedup with dataset size, but remains lower compared to loss computation. This is due to the more complex nature of the gradient kernel, which requires reductions and more intensive memory access.

Method	Dataset	TPB	$S_p G$	$S_p L$
GD CUDA	768x8	256	1.06x	0.81x
CG CUDA	768x8	256	0.97x	0.71x
GD CUDA	1000x24	256	1.03x	0.95x
CG CUDA	1000x24	256	1.05x	0.95x
GD CUDA	1000x60	256	1.08x	0.91x
CG CUDA	1000x60	256	1.21x	0.98x
GD CUDA	1605x119	512	2.46x	1.34x
CG CUDA	1605x119	512	2.87x	1.48x
GD CUDA	3185x122	512	4.71x	3.58x
CG CUDA	3185x122	512	6.11x	4.60x
GD CUDA	11055x68	32	4.44x	18.06x
CG CUDA	11055x68	32	3.99x	16.04x

Table 4. Configurations with best specific speedups

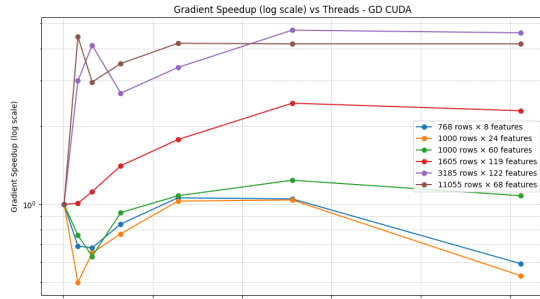


Figure 5. Gradient Descent gradient speedup vs TPB

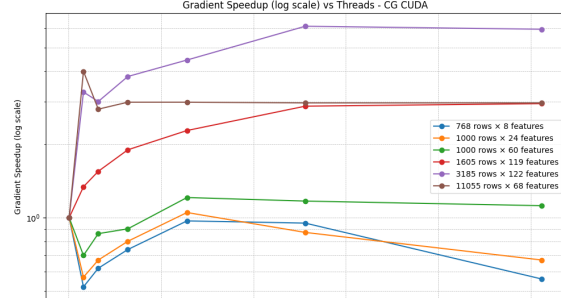


Figure 6. Conjugate Gradient gradient speedup vs TPB

Loss Speedup Analysis Results show that loss computation is highly parallelizable, with speedups up to 18.06x (GD CUDA TPB=32 on 11,055x68), as each sample contributes independently.



Figure 7. Gradient Descent Loss speedup vs TPB

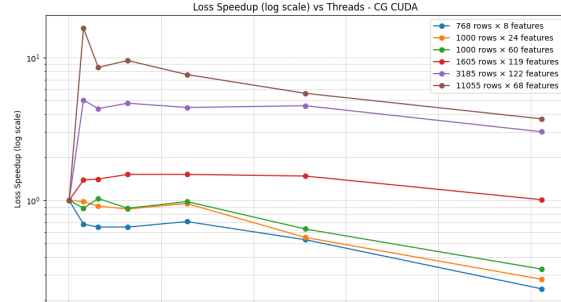


Figure 8. Conjugate Gradient Loss speedup vs TPB

4.3.3 Efficiency

Efficiency is defined as:

$$E = \frac{S_p}{P}$$

where P is the number of active threads. It was calculated separately for gradient and loss.

Total Efficiency: CG CUDA with TPB = 1024 stands out as the most efficient configuration on large datasets, such as 3185x122 and 1605x119, achieving 0.3668 and 0.2318 respectively. GD CUDA shows good performance with TPB = 512, particularly on 3185x122 with 0.2327.

Method	Dataset	TPB	Eff. Tot	Iterations
GD CUDA	768×8	256	0.1044	766
CG CUDA	768×8	512	0.1043	44
GD CUDA	1000×24	512	0.1080	391
CG CUDA	1000×24	512	0.1034	67
GD CUDA	1000×60	512	0.1232	30
CG CUDA	1000×60	512	0.1236	11
GD CUDA	1605×119	512	0.1470	3949
CG CUDA	1605×119	1024	0.2318	149
GD CUDA	3185×122	512	0.2327	7730
CG CUDA	3185×122	1024	0.3668	289
GD CUDA	11055×68	1024	0.2155	4298
CG CUDA	11055×68	1024	0.1455	336

Table 5. Configurations with best total efficiency

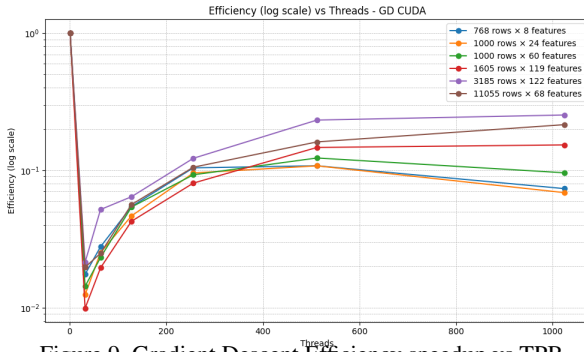


Figure 9. Gradient Descent Efficiency speedup vs TPB

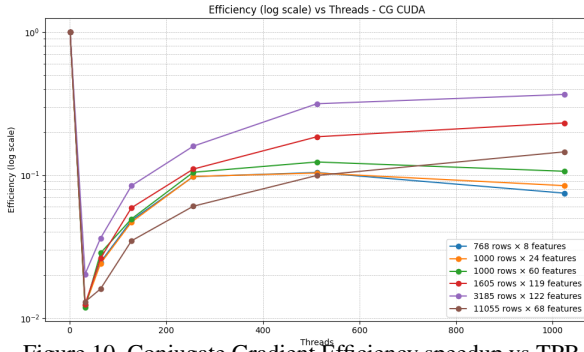


Figure 10. Conjugate Gradient Efficiency speedup vs TPB

Method	Dataset	TPB	Eff. Grad	Eff. Loss
GD CUDA	768×8	256	0.1331	0.1008
CG CUDA	768×8	256	0.1213	0.0889
GD CUDA	1000×24	256	0.1033	0.0948
CG CUDA	1000×24	1024	0.1666	0.0694
GD CUDA	1000×60	512	0.2071	0.1162
CG CUDA	1000×60	1024	0.2812	0.0813
GD CUDA	1605×119	512	0.2461	0.1344
CG CUDA	1605×119	1024	0.4895	0.1689
GD CUDA	3185×122	1024	0.4599	0.2335
CG CUDA	3185×122	1024	0.5944	0.3019
GD CUDA	11055×68	1024	0.1737	0.2216
CG CUDA	11055×68	1024	0.1235	0.1549

Table 6. Configurations with best specific efficiency

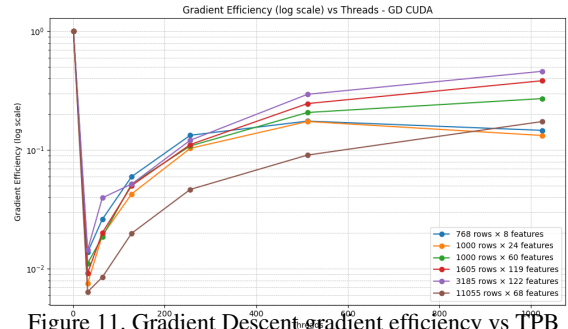


Figure 11. Gradient Descent gradient efficiency vs TPB

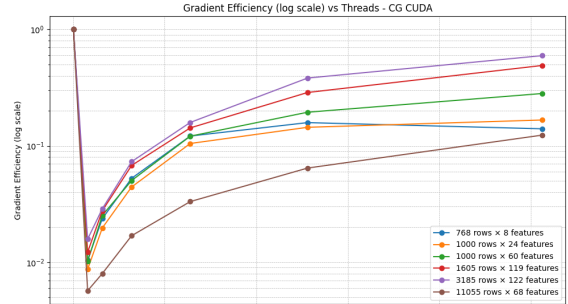


Figure 12. Conjugate Gradient gradient efficiency vs TPB

Gradient Efficiency Analysis Gradient efficiency increases with dataset size and TPB, especially for CG CUDA. The gradient kernel involves reductions and memory-intensive operations, which benefit from higher parallelism. CG CUDA with TPB = 1024 consistently achieves the highest gradient efficiency across datasets, peaking at 0.5944 on 3185 × 122. GD CUDA also shows strong performance, reaching 0.4599 on the same dataset.

Loss Efficiency Analysis Loss computation is highly parallelizable, as each sample contributes independently. This leads to higher speedups and efficiency values, especially for GD CUDA. The highest loss efficiency is observed on 3185 × 122 with CG CUDA TPB = 1024 (0.3019), followed closely by GD CUDA (0.2335). GD CUDA also performs well on 11055 × 68, reaching 0.2216.

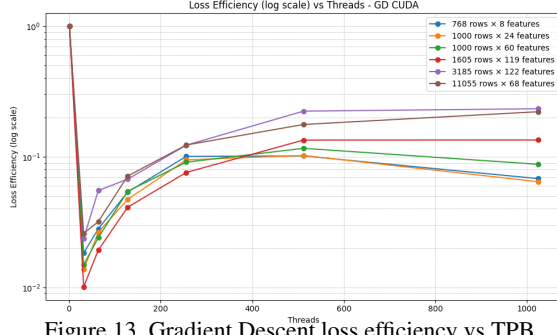


Figure 13. Gradient Descent loss efficiency vs TPB

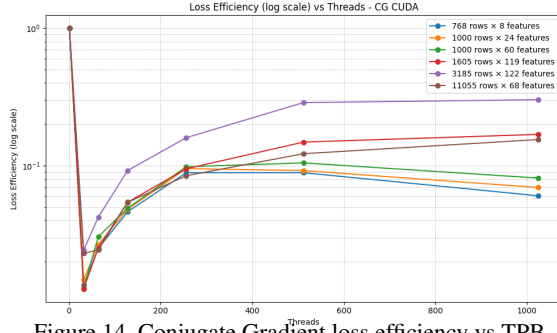


Figure 14. Conjugate Gradient loss efficiency vs TPB

In summary, the results confirm that the choice of TPB and optimization method has a crucial impact on computational efficiency. CG CUDA with TPB = 1024 represents the optimal configuration for maximizing performance on large datasets.

5. Conclusions

CUDA parallelization significantly boosts the performance of optimization algorithms, especially on large datasets. Numba CUDA enabled speedups up to $13.76\times$ overall, $4.44\times$ for gradient computation, and over $18\times$ for loss, demonstrating the benefits of GPU acceleration.

Efficiency varies with dataset size and TPB: smaller datasets suffer from overhead, while larger ones benefit more from parallel execution. However, gains are limited by the non-parallelizable portions of the workload, as described by Amdahl's Law.

Conjugate Gradient consistently outperforms Gradient Descent in convergence speed and efficiency, particularly with TPB = 1024. These results highlight the importance of tuning thread configuration and kernel design to fully exploit parallelism in large-scale optimization.